Beyond90

Football Player Network & Performance Analytics

Metric	Current project value
Players	10
Match records	64
Goals recorded	5
Assists recorded	5
Recorded minutes	5184
Average rating	7.55 / 10

Purpose

Beyond90 is a Django web application for player discovery, profile management, match-performance review, position-aware ranking, and head-to-head analytics. It is designed as an explainable scouting dashboard: every aggregate is derived from match records rather than fabricated values.

Document scope: use, architecture, data model, analytics methodology, dashboard behavior, themes, GitHub extraction, limitations, and local setup.

1. Table of contents

2. Product overview

3. How to use Beyond90

4. Module-by-module guide

5. Data model

6. Analytics methodology

7. Dashboard calculations and filters

8. User interface and themes

9. GitHub account and repository analysis

10. Current extracted dataset summary

11. Setup and operations

12. Validation and limitations

13. Suggested roadmap

2. Product overview

Beyond90 combines three practical views:

Discovery: searchable player cards, public player profiles, club and position information, and profile images.

Performance review: match-level statistics such, as goals, assists, shots, key passes, tackles, interceptions, pass completion, minutes, and rating.

Contextual comparison: filtering, per-90 normalization, position-aware percentiles, selected-player head-to-head output, and recent rating trends.

3. How to use Beyond90

Overview: open the home page to see season totals, top players by goals and rating, recent performances, and total player count.

Players: use the directory to search by name, filter or inspect clubs and positions, then open a player profile for detailed totals and form.

Analytics: choose a date range, exact position, club, competition, and minimum minutes per appearance. Optionally select multiple players to compare. Results only use records satisfying all filters.

Account: sign up to create an associated player profile, edit profile details, and sign out. Public discovery remains available without login.

Theme: use the header theme picker to switch Light, Dark, or Lavender. The selected theme is saved in browser local storage.

4. Module-by-module guide

Module	Responsibility
accounts	Registration, login, logout, profile editing, user-to-player association
players	Player model, player directory, search and player-detail pages
analytics	Match data, overview dashboard, analytics filtering, rankings, comparisons
matches	Match-domain app scaffold for future match management expansion
scouting	Scouting-domain app scaffold for future scout workflow
config	Django settings, root URL routing, ASGI and WSGI configuration

Key application flow

The browser sends a request through **config/urls.py**. Django resolves it to a view in the relevant app. Each view queries the ORM, calculates or aggregates context data, and renders a template from the app's templates directory. Static CSS controls the responsive layouts and theme selection.

5. Data model

Player

Stores name, date of birth, nationality, position, preferred foot, height, club, optional profile image, biography, optional authenticated user relationship, and creation timestamp. A player profile links one-to-one with a Django user account.

MatchPerformance

Stores a foreign key to a player, match date, opponent, competition, minutes played, goals, assists, shots, key passes, tackles, interceptions, pass completion percentage, and a rating from zero to ten. Records are unique by player, date, and opponent.

Integrity behavior

Deleting a player cascades to that player's match records. Players without records have no fabricated analytics and can still appear in the player directory.

6. Analytics methodology

Per-90 normalization

For a filtered set of appearances, each total is normalized using the same recorded minutes: **stat per 90 = stat total × 90 / minutes played**. This makes output comparable between players with different sample lengths, while remaining transparent that it is based on the available records.

Goal contributions

Goal contributions are goals plus assists. The contribution per-90 metric applies the same normalization formula. Dashboard totals are unnormalized sums unless a view explicitly uses per 90.

Averages and pass completion

Average rating is the arithmetic mean of match-rating values in the active scope. Average pass completion is the arithmetic mean of recorded match percentages. Missing or zero values are not treated as additional evidence; templates display them carefully where appropriate.

Position-aware percentiles

Players are grouped into Goalkeeper, Defence, Midfield, and Attack. The benchmark metric is role-specific:

Goalkeepers: average rating; defenders: interceptions per 90; midfielders: key passes per 90; attackers: goals plus assists per 90.

A midpoint-percentile rank is calculated against eligible peers in the same group. With fewer than two peers, the percentile is suppressed rather than misleadingly reported. The same active filters define the peer population.

7. Dashboard calculations and filters

The overview aggregates all match rows for season matches, total goals, assists, average rating, distinct players, and best rating. Leader rankings count appearances, add goals and assists, calculate average rating, then sort by total goals and rating.

The analytics form applies **AND**-logic: records must meet the selected start date, end date, position, club, competition, and minimum minutes criteria. A date range is invalid if the end is before the start.

Player comparison requires at least two matched selected players; selected players excluded by filters are reported as unmatched. Rating trend chart points use the latest ten filtered appearances in chronological order and scale ratings from 5–10 into the chart's SVG vertical range.

8. User interface and themes

The interface uses a calm editorial style with a sticky header, card layouts, restrained typography, player imagery, and responsive tables. The shared stylesheet defines theme variables for ink, background, surface, line, and accent colors.

Light theme: neutral paper colors with green accents. **Dark theme:** dark surfaces with lighter green accents. **Lavender theme:** soft lilac backgrounds with a dark-purple to lavender interaction palette: #4d326f, #7656a8, #9a7bc3, and supporting pale surfaces. Green-only action and highlight classes are remapped through the lavender variables so buttons, focus states, ranking lines, and status dots stay visually consistent.

9. GitHub account and repository analysis

GitHub CLI was authenticated for the account **manasa309**. The account currently exposes 14 repositories listed in the extraction file. Repository metadata, project language data, and the FootballIQ commit history were extracted using GitHub CLI and the GitHub API. The commit export is retained as an analysis artifact alongside this project session.

No tabular CSV or XLSX dataset was discovered in the account's searchable public code index, so the extracted primary dataset is the application's own SQLite database rather than inventing external football records. GitHub repository data is used to document project provenance, implementation language mix, and account-level work history; it is not blended into football statistics.

10. Current extracted dataset summary

Current SQLite totals

Metric	Current project value
Players	10
Match records	64
Goals recorded	5
Assists recorded	5
Recorded minutes	5184
Average rating	7.55 / 10

Players by coded position

Position	Player count
AM	1
CB	1
CM	1
DM	1
GK	1
LB	1
LW	1
RB	1
ST	2

Players by club

Club	Player count
Eastvale Rovers	1
Harbor City FC	2
Lakeside United	1
Northbridge FC	2
Redwood Athletic	2
real madrid	2

Data-quality note

The seeded database contains illustrative demo records if the demo seed command has been run. It is not confirmed third-party match data. Do not publish seeded records as real-world statistics without replacing them with licensed or verified source data.

11. Setup and operations

Create a virtual environment, install **requirements.txt**, run migrations, optionally run the demo data seeder, create a superuser, and start the development server. The README documents these commands. Use Django admin to maintain players and match records.

Recommended workflow: backup the database before bulk edits; use Django migrations for schema changes; validate calculations after model or view changes; keep demo and production data clearly separated; and document any external data provider.

12. Validation and limitations

The project passed Django's system check and migrations were found to be current at document generation time. The current database contained 10 players and 64 match records. Automated test discovery found no project tests in the referenced test modules.

Important limitations:

1. Percentiles are descriptive within the existing project dataset and are not predictive.

2. Per-90 rates can exaggerate output from very small minute samples.

3. Ratings and pass percentages depend on source methodology and are not standardized external scouting grades unless documented.

4. The dashboard does not adjust for opponent strength, home/away, substitutions, team tactics, position transitions, injury time, or expected-goals models.

5. Searchable analysis is scoped to stored records and does not import live football feeds.

13. Suggested roadmap

Recommended improvements for a stronger analytical product:

Add source provenance and season fields to match records; support event-level data and opponent ratings; add confidence intervals and minimum sample thresholds; separate public demo data from verified production data; add automated unit tests for all aggregation formulas; version percentile definitions; and introduce data-import validation with CSV, API, and database lineage audit logs.

End of Beyond90 project documentation